GUIDE DE RÉVISION — ORAL TECHNIQUE

MAISON XCIV

Application e-commerce de montres de luxe

Node.js · Express · JWT · JSON Database

ÉTUDIANT	DATE	STACK
	2026	Node.js v24 · Express · JWT

Architecture REST · Authentification JWT · Middleware Express · CRUD complet · Cookies HttpOnly · Upload d'images · Module Facturation · Favoris temps réel

Table des matières

1. Architecture globale du projet
2. Stack Technique
3. Système d'authentification
4. API REST complète
5. Base de données JSON
6. Frontend — Pages & Fonctions JS
7. Module Facturation
8. Questions d'oral & Réponses
9. Cartes mentales des flux

01 Architecture globale du projet

Maison XCIV est une application web **fullstack** organisée en deux grandes parties : un **frontend** statique servi par Express, et un **backend** Node.js/Express exposant une API REST. Les données sont persistées dans des **fichiers JSON**.

```
/maison-xciv-site ├── /frontend → Tout ce que voit l'utilisateur │ ├── index.html → Site public (catalogue montres) │ ├── style.css → Styles globaux │ ├── script.js → JS : auth, favoris, modal, nav │ ├── /admin → Interface administrateur │ │ ├── login.html │ │ ├── dashboard.html │ │ ├── admin.css │ │ └── facturation.html │ ├── /connexion → Login + Inscription │ │ ├── index.html → Login user │ │ ├── inscription.html │ │ └── auth.css │ ├── /mon-espace → Espace client connecté │ │ ├── index.html │ │ └── profil.js │ └── /assets/images/uploads │ ├── /backend → Serveur Node.js / Express │ ├── server.js → Point d'entrée, config Express │ ├── /config │ │ ├── admin.js → Credentials admin + JWT secret admin │ │ └── userConfig.js → JWT secret user + règles cookies │ ├── /middleware │ │ └── auth.js → 4 middleware d'authentification │ ├── /controllers → Logique métier │ │ ├── watchesController.js │ │ ├── authController.js │ │ ├── userController.js │ │ ├── favoritesController.js │ │ └── invoicesController.js │ ├── /models → Accès aux données │ │ ├── watchModel.js │ │ ├── userModel.js │ │ ├── favoriteModel.js │ │ └── invoiceModel.js │ └── /routes → Définition des endpoints │ ├── watches.js → /api/watches/* │ ├── auth.js → /api/auth/* │ ├── users.js → /api/user/* │ ├── favorites.js → /api/favorites/* │ ├── upload.js → /api/upload │ └── invoices.js → /api/admin/invoices/* │ ├── /database → Persistance JSON │ ├── database.js + watches.json │ ├── usersDB.js + users.json │ ├── favoritesDB.js+ favorites.json │ └── invoicesDB.js+ invoices.json │ └── package.json → express, cors, jsonwebtoken, bcryptjs, multer...
```

Responsabilités de chaque couche

Routes (/routes)

Définissent **quelles URL existent** et quel middleware s'applique. Elles transmettent la requête aux controllers. Ne contiennent pas de logique métier.

Controllers (/controllers)

Contiennent la **logique métier** : valider les données, appeler les models, construire la réponse JSON. Le cerveau de l'API.

Models (/models)

Fonctions d'**accès aux données** : lire, écrire, modifier les fichiers JSON. Les models ne savent rien d'Express.

Middleware (/middleware)

Code qui s'exécute **entre la requête et le controller**. Vérifie l'authentification, peut bloquer ou laisser passer.

Frontend (/frontend)

Pages HTML + CSS + JS servis **statiquement** par Express. Communique avec l'API via `fetch()`.

Database (/database)

Modules Node.js qui **lisent/écrivent les fichiers JSON** avec `fs.readFileSync` / `fs.writeFileSync`. Simule une vraie DB.

Flux d'une requête HTTP

Client (navigateur)
fetch('/api/watches')

↓

Express Router
watches.js — route matching

↓

Middleware Auth
optionalUserAuth / requireAuth
Controller
watchesController.js — logique
Model + JSON DB
watchModel.js → watches.json
Réponse JSON
res.json({data})

02 Stack Technique

PACKAGE	RÔLE	EXEMPLE D'UTILISATION
`express`	Framework web Node.js — gère routes, middleware, requêtes HTTP	`app.get('/api/watches', handler)`
`jsonwebtoken`	Créer et vérifier des tokens JWT signés	`jwt.sign(payload, secret, {expiresIn})`
`bcryptjs`	Hacher les mots de passe de façon sécurisée (jamais stocker en clair)	`bcrypt.hash(password, 10)`
`cookie-parser`	Lire et écrire des cookies HTTP (dont HttpOnly)	`res.cookie('xciv_user_token', jwt, opts)`
`multer`	Gérer l'upload de fichiers (images des montres)	`multer.diskStorage({destination, filename})`
`cors`	Autoriser les requêtes cross-origin (navigateur → API)	`app.use(cors({origin, credentials: true}))`
`nodemon`	Redémarre auto le serveur à chaque modification de fichier (dev only)	`npm run dev`

JWT — JSON Web Token

Un token JWT est composé de **3 parties séparées par des points** :

- **Header** : algorithme (HS256)
- **Payload** : données (id, email, rôle)
- **Signature** : HMAC signé avec le secret

> **Analogie :** comme un badge d'accès tamponné par l'entreprise. N'importe qui peut lire le nom dessus, mais seule l'entreprise (qui a le tampon secret) peut en créer un valide.

bcrypt — Hachage de mots de passe

bcrypt ne **chiffre pas** (réversible), il **hache** (irréversible). Il ajoute un *salt* aléatoire pour que deux mots de passe identiques donnent des hashes différents.

- Le coût (10 rounds) rend le brute-force lent
- `bcrypt.compare(plaintext, hash)` → vrai/faux
- On ne peut jamais "récupérer" le mot de passe

Cookies HttpOnly

Un cookie `HttpOnly` est **inaccessible depuis JavaScript** (protection contre XSS). Il est envoyé automatiquement par le navigateur sur chaque requête.

- `Secure` : uniquement en HTTPS
- `SameSite: Strict` : protection CSRF
- Le JS ne peut pas faire `document.cookie` pour le lire

CORS — Cross-Origin Resource Sharing

Par défaut, un navigateur bloque les requêtes vers un **domaine différent**. CORS permet d'autoriser explicitement certaines origines.

- `credentials: true` → autorise les cookies cross-origin
- L'origine doit être déclarée explicitement (pas `*`)
- Le serveur ajoute des headers `Access-Control-Allow-*`

Configuration server.js — Points clés

```
const express = require('express');
const app = express();

// Middleware globaux
app.use(express.json());                    // Lire req.body JSON
```

```
app.use(cookieParser());                  // Lire req.cookies
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));

// Servir le frontend statiquement
app.use(express.static('frontend'));

// Monter les routes API
app.use('/api/watches', watchRoutes);
app.use('/api/auth',    authRoutes);
app.use('/api/user',    userRoutes);
app.use('/api/favorites', favoriteRoutes);
app.use('/api/admin/invoices', invoiceRoutes);

app.listen(3000);
```

03 Système d'authentification

Principe clé : Le projet utilise **2 systèmes d'auth séparés** avec 2 cookies distincts et 2 secrets JWT différents. L'admin et l'utilisateur ne partagent aucun token.

Admin — xciv_admin_token

POST /api/auth/login
email + password

↓

bcrypt.compare()
avec hash dans admin.js

↓

jwt.sign(payload, ADMIN_SECRET)
expiration : 8h

↓

Cookie HttpOnly
xciv_admin_token

User — xciv_user_token

POST /api/user/login
email + password

↓

bcrypt.compare()
avec hash dans users.json

↓

jwt.sign(payload, USER_SECRET)
expiration : 7j

↓

Cookie HttpOnly
xciv_user_token

Les 4 middleware d'authentification

`requireAuth`

Vérifie le cookie admin. Si invalide → `401` JSON ou redirect vers login admin. Protège toutes les routes d'administration.

`requireAuthPage`

Comme requireAuth mais pour les pages HTML. Redirige vers `/admin/login` au lieu de renvoyer du JSON.

`requireUserAuth`

`optionalUserAuth`

Vérifie le cookie user. Si invalide → `401` JSON. Protège `/api/favorites`, `/api/user/me`.

Tente de lire le token user. Succès → peuple `req.user`. Échec → passe quand même. **Ne bloque jamais.** Utilisé sur le site public.

Exemple de code — middleware auth.js

```
// optionalUserAuth — ne bloque JAMAIS, essentiel pour le site public
const optionalUserAuth = (req, res, next) => {
  const token = req.cookies['xciv_user_token'];
  if (!token) return next(); // Pas de cookie → visiteur anonyme, on continue
  try {
    req.user = jwt.verify(token, USER_SECRET);
    next(); // Token valide → req.user dispo dans le controller
  } catch {
    next(); // Token invalide/expiré → on continue quand même
  }
};

// requireUserAuth — bloque si non connecté
const requireUserAuth = (req, res, next) => {
  const token = req.cookies['xciv_user_token'];
  if (!token) return res.status(401).json({ error: 'Non authentifié' });
  try {
    req.user = jwt.verify(token, USER_SECRET);
    next();
  } catch {
    return res.status(401).json({ error: 'Token invalide' });
  }
};
```

04 API REST complète

Convention REST : Les méthodes HTTP ont une sémantique précise — GET = lire, POST = créer, PUT = modifier, DELETE = supprimer. L'URL identifie la ressource, la méthode l'action.

Routes publiques (aucune auth)

MÉTHODE	ENDPOINT	DESCRIPTION
GET	/api/watches	Récupérer tout le catalogue de montres
GET	/api/watches/:id	Récupérer une montre par son ID
GET	/api/user/session	État de la session (renvoie null si non connecté, jamais d'erreur)
POST	/api/user/register	Créer un compte utilisateur
POST	/api/user/login	Connexion utilisateur (crée cookie)
POST	/api/user/logout	Déconnexion utilisateur (efface cookie)

Routes admin (requireAuth — cookie xciv_admin_token)

MÉTHODE	ENDPOINT	DESCRIPTION
POST	/api/auth/login	Login admin (bcrypt + JWT admin)
POST	/api/auth/logout	Logout admin (efface cookie)
GET	/api/auth/verify	Vérifier si le token admin est valide
POST	/api/watches	Créer une nouvelle montre

MÉTHODE	ENDPOINT	DESCRIPTION
PUT	/api/watches/:id	Modifier une montre existante
DELETE	/api/watches/:id	Supprimer une montre
POST	/api/upload	Upload image (multer → /assets/images/uploads)
GET	/api/admin/invoices	Liste factures avec filtres (search/status/dates)
GET	/api/admin/invoices/:id	Détail d'une facture
POST	/api/admin/invoices	Créer une facture
PUT	/api/admin/invoices/:id	Modifier une facture
DELETE	/api/admin/invoices/:id	Supprimer une facture
POST	/api/admin/invoices/:id/duplicate	Dupliquer une facture
GET	/api/admin/invoices/stats	Statistiques globales
GET	/api/admin/invoices/next-number	Prochain numéro de facture disponible
GET	/api/admin/invoices/clients-summary	Synthèse par client
GET	/api/admin/invoices/export/csv	Export CSV (avec filtres actifs)

Routes user connecté (requireUserAuth — cookie xciv_user_token)

MÉTHODE	ENDPOINT	DESCRIPTION
GET	/api/user/me	Profil de l'utilisateur connecté
GET	/api/favorites	Liste des montres en favoris

MÉTHODE	ENDPOINT	DESCRIPTION
GET	/api/favorites/ids	Seulement les IDs des favoris
POST	/api/favorites/:id	Ajouter/retirer un favori (toggle)

Exemple de requête / réponse JSON

```
// POST /api/user/login — Requête
{
  "email":    "alice@example.com",
  "password": "monMotDePasse123"
}

// Réponse 200 — Le cookie est créé côté serveur (Set-Cookie header)
{
  "success": true,
  "user": {
    "id":    "abc123",
    "email": "alice@example.com",
    "name":  "Alice"
  }
}

// GET /api/watches — Réponse
{
  "watches": [
    { "id": "w1", "name": "Submariner", "price": 15000, "brand": "Rolex" }
  ]
}
```

05 Base de données JSON

Choix architectural : Pas de SQLite ni de PostgreSQL car le projet tourne sur Windows sans les Visual C++ Build Tools (requis pour compiler sqlite3). Les fichiers JSON remplacent la DB avec un pattern simple : lire tout → modifier → réécrire.

watches.json

```
[
  {
    "id":       "w1",
    "name":     "Submariner",
    "brand":    "Rolex",
    "price":    15000,
    "promo":    12000,
    "image":    "/assets/...",
    "category": "Plongée",
    "stock":    3
  }
]
```

users.json

```
[
  {
    "id":        "u1",
    "email":     "alice@ex.com",
    "password":  "$2b$10$...",
    "name":      "Alice",
    "createdAt": "2025-01-15"
  }
]
```

Le mot de passe est toujours un hash bcrypt, jamais en clair.

favorites.json

```
[
  {
    "userId":  "u1",
    "watchId": "w1"
  },
  {
    "userId":  "u1",
    "watchId": "w3"
  }
]
```

Table de jointure simple (userId ↔ watchId).

invoices.json

```
[
  {
    "id":       "inv_001",
    "number":  "XCIV-2025-001",
    "client":  { "name": "..." },
    "items":   [],
    "total":   15000,
    "status":  "paid"
  }
]
```

Pattern de chaque module DB (ex: database.js)

```
const fs   = require('fs');
const path = require('path');

const DB_PATH = path.join(__dirname, 'watches.json');

// Lire toutes les montres (synchrone car simple)
function readWatches() {
  return JSON.parse(fs.readFileSync(DB_PATH, 'utf8'));
}

// Écrire (remplace tout le fichier)
function writeWatches(watches) {
  fs.writeFileSync(DB_PATH, JSON.stringify(watches, null, 2));
}

// Exemple : ajouter une montre
function addWatch(watch) {
  const watches = readWatches();
  watches.push(watch);
  writeWatches(watches);
  return watch;
}

module.exports = { readWatches, writeWatches, addWatch };
```

Limite de l'approche JSON : Pas de transactions, pas de requêtes complexes, tout le fichier est relu à chaque opération. Suffit pour un projet de démonstration ; en production on utiliserait PostgreSQL ou MongoDB.

06 Frontend — Pages & Fonctions JS clés

/ — index.html

Site public. Catalogue de montres. Navigation dynamique selon auth. Cœurs favoris. Modal de connexion.

- `initAuth()`
- `loadWatches()`
- `buildCard()`
- `toggleFavorite()`

/connexion — Login

Formulaire de connexion. Envoie POST /api/user/login. Redirige vers /mon-espace si succès.

- `handleLogin()`
- Validation frontend

/inscription — Register

Formulaire d'inscription. Valide les règles (longueur password, format email). POST /api/user/register.

/mon-espace — Espace client

Affiche les favoris de l'utilisateur connecté. `profil.js` appelle GET /api/favorites et construit les cartes.

/admin — Dashboard admin

CRUD montres complet. Upload image. Tableau de bord stats. Module facturation SPA.

- Toutes les actions passent par l'API admin

Flux du chargement de la page d'accueil

DOMContentLoaded

Le DOM est prêt

Fonctions JS clés — script.js

```
// 1. initAuth — premier appel, doit finir AVANT loadWatches
async function initAuth() {
  const res  = await fetch('/api/user/session', { credentials: 'include' });
  const data = await res.json();
  currentUser  = data.user;      // null si visiteur
  favoriteIds  = data.favoriteIds || []; // [] si non connecté
  updateNavAuth(currentUser);
}

// 2. buildCard — injecte le cœur avec état pré-calculé
function buildCard(watch, ids) {
  const isFav = ids.includes(watch.id);
  return `<div class="card">
    <button class="heart ${isFav ? 'active' : ''}"
            onclick="toggleFavorite('${watch.id}')">♥</button>
    <img src="${watch.image}" />
    <h3>${watch.name}</h3>
  </div>`;
}

// 3. toggleFavorite — POST avec gestion 401
async function toggleFavorite(id) {
  const res = await fetch(`/api/favorites/${id}`, {
```

```
    method: 'POST', credentials: 'include'
  });
  if (res.status === 401) return showLoginModal(); // Visiteur → modal
  // Mettre à jour favoriteIds et reconstruire le cœur
}
```

IntersectionObserver — Apparition animée des cartes

```
// Déclenche l'animation d'entrée quand une carte devient visible
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      entry.target.classList.add('visible');
      observer.unobserve(entry.target); // Une seule fois suffit
    }
  });
}, { threshold: 0.1 }); // Se déclenche à 10% de visibilité

document.querySelectorAll('.card').forEach(c => observer.observe(c));
```

07 Module Facturation

Le module de facturation est une **SPA (Single Page Application)** intégrée dans le dashboard admin. Il permet la gestion complète du cycle de vie des factures sans rechargement de page.

CRUD Factures

- Créer avec numérotation auto (XCIV-YYYY-NNN)
- Modifier statut (brouillon, envoyée, payée, annulée)
- Supprimer avec confirmation
- Dupliquer (gain de temps)

Filtres avancés

- Recherche textuelle (client, numéro)
- Filtre par statut
- Filtre par statut de paiement
- Filtre par plage de dates

Statistiques

- Total facturé (toutes périodes)
- Montant en attente
- Nombre de factures par statut
- Synthèse par client

Export CSV

- Respecte les filtres actifs
- Généré côté serveur
- Headers : Content-Type text/csv
- Téléchargement direct navigateur

Exemple — Export CSV dans le controller

```
// invoicesController.js — exportCSV
async function exportCSV(req, res) {
  const invoices = invoiceModel.getAll(req.query); // Filtres passés en query

  const header = 'Numéro,Client,Date,Montant,Statut\n';
  const rows   = invoices.map(i ⇒
    `${i.number},${i.client.name},${i.date},${i.total},${i.status}`
  ).join('\n');

  res.setHeader('Content-Type', 'text/csv');
  res.setHeader('Content-Disposition', 'attachment; filename=factures.csv');
```

```
  res.send(header + rows);
}
```

Upload d'images (multer)

```
// upload.js — Configuration multer
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'frontend/assets/images/uploads'); // Dossier de destination
  },
  filename: (req, file, cb) => {
    const unique = Date.now() + '-' + Math.round(Math.random() * 1e9);
    cb(null, unique + path.extname(file.originalname)); // Nom unique
  }
});
const upload = multer({ storage, limits: { fileSize: 5 * 1024 * 1024 } }); // 5 MB ma

router.post('/', requireAuth, upload.single('image'), (req, res) => {
  res.json({ url: `/assets/images/uploads/${req.file.filename}` });
});
```

08 Questions d'oral & Réponses

Q1 Pourquoi avoir choisi JSON et pas SQLite comme base de données ?

SQLite nécessite de compiler un module natif C++ lors de `npm install`, ce qui requiert les **Visual C++ Build Tools** sur Windows — non installés sur la machine de développement. Plutôt que de bloquer le projet sur une configuration système complexe, les fichiers JSON avec le module `fs` de Node.js offrent une solution **zéro dépendance native**, simple à comprendre et suffisante pour les volumes de données d'un portfolio. Le code reste lisible et portable.

> Analogie : c'est comme utiliser une feuille Excel au lieu d'un vrai SGBD pour gérer 100 clients. Ca fonctionne parfaitement à cette échelle.

Q2 Comment fonctionne JWT (JSON Web Token) ?

Un JWT est composé de **3 parties encodées en Base64 et séparées par des points** : `header.payload.signature`. Le serveur crée le token avec `jwt.sign(payload, secret)`, le signe avec une clé secrète (HMAC SHA-256), et l'envoie au client. À chaque requête suivante, le client renvoie le token. Le serveur vérifie la signature avec `jwt.verify(token, secret)` — si quelqu'un a modifié le payload, la signature ne correspond plus et le token est rejeté. **Le serveur ne stocke rien** : le token est autoporteur.

> Analogie : c'est comme un passeport. Le pays émetteur (serveur) appose un tampon (signature). Chaque frontière (endpoint) vérifie l'authenticité du tampon sans appeler le pays d'origine.

Q3 Pourquoi 2 cookies séparés (admin vs user) ?

Pour des raisons de **sécurité et de séparation des responsabilités**. Les deux rôles ont des durées de vie différentes (admin 8h, user 7j), des secrets JWT différents, et des niveaux de privilège complètement distincts. Si le token user était compromis, il ne donnerait aucun accès admin. Les middleware peuvent vérifier indépendamment chaque token. Cette architecture suit le **principe du moindre privilège**.

Q4 Qu'est-ce que bcrypt et pourquoi l'utiliser ?

bcrypt est un algorithme de **hachage de mots de passe**. Contrairement à MD5 ou SHA-1 qui sont des fonctions rapides, bcrypt est **intentionnellement lent** (paramétrable avec un "cost factor") pour rendre le brute-force difficile. Il intègre automatiquement un **salt aléatoire**, ce qui signifie que deux mots de passe identiques produisent des hashes différents (protection contre les rainbow tables). On ne stocke jamais le mot de passe en clair, seulement son hash. `bcrypt.compare()` rehache le mdp fourni et compare.

> Analogie : bcrypt est comme un coffre-fort dont l'ouverture prend intentionnellement 10 secondes. Pas gênant pour un vrai utilisateur, mais catastrophique pour un attaquant qui essaie un milliard de mots de passe.

Q5 Comment fonctionne optionalUserAuth ?

C'est un middleware Express qui **essaie** de lire et valider le cookie `xciv_user_token`. Si le cookie est absent ou invalide, il appelle `next()` quand même — la requête continue. Si le token est valide, il peuple `req.user` avec les données décodées. L'avantage : le site public fonctionne pour tout le monde (visiteurs anonymes et connectés), mais les fonctions qui utilisent `req.user` peuvent personnaliser l'expérience si l'utilisateur est connecté (ex: pré-calculer les favoris pour `buildCard()`).

Q6 Qu'est-ce qu'un middleware Express ?

Un middleware est une **fonction intermédiaire** qui s'exécute entre la réception d'une requête HTTP et l'envoi de la réponse. Sa signature est `(req, res, next)`. Il peut modifier `req` (ex: ajouter `req.user`), répondre directement (bloquer), ou appeler `next()` pour passer au middleware/handler suivant. Les middleware s'enchaînent : `app.use(cors())` → `app.use(cookieParser())` → route handler. C'est la colonne vertébrale d'Express.

> Analogie : comme des agents de sécurité à l'entrée d'un club. Le premier vérifie l'âge (cors), le second vérifie les cookies (cookieParser), le troisième vérifie le token (auth). Si tous passent, vous entrez dans la salle (controller).

Q7 Comment multer gère l'upload d'images ?

multer est un middleware Express qui intercepte les requêtes `multipart/form-data` (le format utilisé pour les uploads). On configure `multer.diskStorage()` avec deux fonctions : `destination` (quel dossier) et `filename` (comment nommer le fichier). `upload.single('image')` indique qu'on attend un seul fichier avec le champ "image". Après traitement, `req.file` contient les métadonnées du fichier sauvegardé (nom, chemin, taille...).

Q8 Qu'est-ce que CORS et pourquoi est-ce nécessaire ?

CORS (Cross-Origin Resource Sharing) est un mécanisme de sécurité des navigateurs qui **bloque les requêtes HTTP vers un domaine différent** de la page courante (par défaut). Quand le frontend (port 3000) fait un `fetch` vers l'API (même port ici, mais en général différent), le navigateur vérifie que le serveur autorise cette origine via les headers `Access-Control-Allow-Origin`. Avec `credentials: true`, l'origine doit être explicite (pas `*`) pour permettre l'envoi de cookies.

Q9 Comment les favoris sont-ils stockés et gérés ?

Les favoris sont stockés dans `favorites.json` comme une **table de jointure plate** : un tableau d'objets `{userId, watchId}`. Quand un utilisateur clique sur un cœur, le frontend envoie `POST /api/favorites/:watchId`. Le controller vérifie si la paire (userId, watchId) existe déjà : si oui, il la supprime (unfavorite) ; sinon, il l'ajoute (favorite). C'est le pattern **toggle**. Pour afficher les cœurs à la page, `GET /api/favorites/ids` retourne uniquement les IDs — plus léger qu'envoyer toutes les montres.

Q10 Différence entre requireAuth et requireUserAuth ?

requireAuth protège les routes **admin** : vérifie le cookie `xciv_admin_token` signé avec le secret admin. Si invalide → 401 ou redirect vers /admin/login. Utilisé sur toutes les routes de gestion (CRUD montres, factures, upload).

requireUserAuth protège les routes **utilisateur connecté** : vérifie `xciv_user_token` avec le secret user. Si invalide → 401 JSON. Utilisé sur /api/favorites et /api/user/me. Les deux middleware sont totalement indépendants.

Q11 Qu'est-ce qu'un cookie HttpOnly et pourquoi l'utiliser ?

Un cookie `HttpOnly` est un cookie que **JavaScript ne peut pas lire** (`document.cookie` ne le voit pas). Il est transmis automatiquement par le navigateur dans chaque requête HTTP, mais reste inaccessible au code JS de la page. Cela protège contre les attaques **XSS (Cross-Site Scripting)** : même si un attaquant injecte du JS malveillant dans la page, il ne peut pas voler le token JWT stocké dans le cookie.

> Analogie : c'est comme un badge d'accès que vous portez mais que vous ne pouvez pas voir ni donner à quelqu'un — seul le lecteur de badge (le serveur) peut l'utiliser.

Q12 Comment le frontend sait si l'utilisateur est connecté ?

Au chargement de la page, `initAuth()` appelle `GET /api/user/session` avec `credentials: 'include'` (pour envoyer les cookies). Le serveur lit le cookie `xciv_user_token` via `optionalUserAuth` et renvoie `{user: {...}, favoriteIds: [...]}` si connecté, ou `{user: null, favoriteIds: []}` si non connecté. **Cette route ne renvoie jamais d'erreur** — le JS peut toujours lire la réponse. Ensuite, la nav est mise à jour et les cœurs sont colorés selon `favoriteIds`.

Q13 Pourquoi nodemon en développement ?

nodemon surveille les fichiers du projet et **redémarre automatiquement le serveur Node.js** à chaque modification. Sans nodemon, il faudrait arrêter (`Ctrl+C`) et relancer manuellement `node server.js` après chaque changement de code. nodemon est une dépendance de développement (`devDependencies`) — il n'est pas installé en production. D'où les deux scripts : `npm start` → `node server.js`, `npm run dev` → `nodemon server.js`.

Q14 Comment fonctionne l'export CSV ?

Le controller `exportCSV` récupère les factures en appliquant les mêmes filtres que l'affichage (passés en query string). Il construit une chaîne CSV manuellement (header + lignes). Il configure la réponse HTTP avec les headers appropriés : `Content-Type: text/csv` et `Content-Disposition: attachment; filename=factures.csv`. Ce dernier header indique au navigateur de **télécharger le fichier** plutôt que de l'afficher. Le navigateur crée automatiquement un fichier .csv sur l'ordinateur de l'utilisateur.

Q15 Qu'est-ce que le pattern MVC ?

MVC signifie **Model — View — Controller**. C'est un pattern d'architecture qui sépare les responsabilités :

- **Model** : accès aux données (models/ dans ce projet, lit/écrit les JSON)
- **View** : présentation (frontend HTML/CSS/JS)
- **Controller** : logique métier (controllers/, orchestre Model et View)

Cette séparation facilite la maintenance : changer la DB (ex: passer à PostgreSQL) ne touche que les Models. Changer l'UI ne touche que le Frontend.

> Analogie : comme un restaurant. Le Model est la cuisine (données brutes), la View est la salle (présentation), le Controller est le serveur (fait le lien).

Q16 Comment Express route les requêtes ?

Express parcourt les routes dans l'ordre où elles sont définies. Quand une requête arrive, il cherche la première route dont la **méthode HTTP et le chemin correspondent**. Les routeurs (`express.Router()`) permettent de grouper des routes et de les monter sur un préfixe. Ex : `app.use('/api/watches', watchRouter)` → toutes les routes du watchRouter sont prefixées par `/api/watches`. Les paramètres dynamiques (`:id`) sont accessibles via `req.params.id`.

Q17 Qu'est-ce que bcryptjs.compare() fait exactement ?

`bcrypt.compare(plaintext, hash)` ne "déchiffre" pas le hash (c'est impossible). Il **rehache le mot de passe en clair** avec le même salt que celui intégré dans le hash stocké (le salt fait partie du hash bcrypt), puis compare les deux résultats. Si le mot de passe est identique à celui qui a produit le hash, les résultats correspondent. La fonction retourne une promesse qui résout à `true` ou `false`. La comparaison est en **temps constant** (protection contre les timing attacks).

Q18 Comment le JWT est-il vérifié côté serveur ?

`jwt.verify(token, secret)` effectue 3 vérifications :
1. **Intégrité** : re-signe le header + payload avec le secret et compare à la signature du token. Si différent → rejet.
2. **Expiration** : vérifie le champ `exp` du payload. Si date passée → `TokenExpiredError`.
3. **Algorithme** : vérifie que l'algorithme déclaré dans le header est celui attendu.

Si toutes les vérifications passent, il retourne le payload décodé. Sinon il lève une exception catchée dans les middleware.

Q19 Qu'est-ce que multer.diskStorage ?

`multer.diskStorage()` configure comment multer stocke les fichiers uploadés **directement sur le disque** (par opposition à `memoryStorage` qui garde le fichier en RAM). On lui passe un objet avec 2 callbacks :
- `destination(req, file, cb)` : détermine le dossier de sauvegarde. On appelle `cb(null, 'chemin/du/dossier')`.
- `filename(req, file, cb)` : détermine le nom du fichier. On génère un nom unique (Date.now + random) pour éviter les collisions.

Q20 Comment fonctionne IntersectionObserver ?

`IntersectionObserver` est une API native du navigateur qui observe si un élément **entre dans le viewport** (zone visible de l'écran). On crée un observer avec un callback et un seuil (`threshold: 0.1` = se déclenche quand 10% de l'élément est visible). On l'attache à des éléments avec `observer.observe(element)`. Quand la condition est remplie, le callback reçoit un tableau d' `IntersectionObserverEntry`. On ajoute une classe CSS pour déclencher l'animation, puis `unobserve()` pour ne pas répéter. **C'est plus performant que d'écouter l'événement scroll.**

> Analogie : comme une caméra de surveillance qui notifie quand quelqu'un entre dans son champ de vision — au lieu de regarder en permanence si la personne est là.

Q21 Comment fonctionne l'inscription utilisateur côté serveur ?

1. `POST /api/user/register` reçoit `{email, password, name}`
2. Le controller vérifie que l'email n'existe pas déjà dans `users.json`
3. Validation des règles (longueur mdp, format email)
4. `bcrypt.hash(password, 10)` → hash du mot de passe
5. Création de l'objet user avec un ID unique (`Date.now()` ou `crypto.randomUUID()`)
6. Sauvegarde dans `users.json`
7. Création du JWT et du cookie `xciv_user_token`
8. Réponse 201 avec les infos user (sans le hash !)

Q22 Pourquoi initAuth() doit être appelé AVANT loadWatches() ?

`initAuth()` récupère la liste des `favoriteIds` de l'utilisateur connecté. `loadWatches()` → `buildCard()` utilise ces `favoriteIds` pour colorer les cœurs en rouge dès le premier affichage. Si `loadWatches()` était appelé avant, les cœurs seraient tous vides (favoriteIds = []), et il faudrait re-rendre toutes les cartes après. En attendant la promesse d' `initAuth()` avec `await`, on s'assure que les données sont prêtes. C'est une question d'**ordre d'exécution asynchrone**.

09 Cartes mentales des flux

Flux d'une requête admin (ex: créer une montre)

POST /api/watches (admin)

1. Navigateur
- fetch POST /api/watches
- credentials: 'include'
- body: JSON montre
- Cookie auto-envoyé

2. Express Router
- watches.js
- POST '/' route match
- Appel middleware

3. requireAuth
- Lit xciv_admin_token
- jwt.verify() admin
- OK → next()
- KO → 401 JSON

4. watchesController
- Valide les données
- Génère un ID unique
- Appelle watchModel.add()

5. watchModel + JSON
- readWatches()
- watches.push(new)
- writeWatches()

6. Réponse
- res.status(201)
- res.json(newWatch)
- Frontend refresh

Flux inscription utilisateur

POST /api/user/register

1. Formulaire
- Validation JS frontend
- email + password + name
- fetch POST

2. userController
- Email existe déjà ?
- Règles password OK ?
- Si erreur → 400

3. bcrypt.hash()
- 10 rounds de salage
- Produit $2b$10$...
- Irréversible

4. userModel.create()

- Génère ID unique
- Ajoute à users.json
- Retourne user (sans hash)

5. jwt.sign()

- Payload: id, email
- Secret USER_SECRET
- Expire en 7j

6. Cookie + Réponse

- Set-Cookie HttpOnly
- xciv_user_token
- 201 + redirect /mon-espace

Flux toggle favori (cœur)

Clic sur le cœur ♥

1. toggleFavorite(id)

- Appel JS frontend
- POST /api/favorites/:id
- credentials: include

2. requireUserAuth

- Lit xciv_user_token
- Non connecté → 401
- 401 → showLoginModal()

3. favoritesController

- req.user.id disponible
- Cherche (userId, watchId)
- Existe ? → supprime
- Non ? → ajoute

4. favoriteModel

- Lit favorites.json
- filter() ou push()
- Réécrit le fichier

5. Réponse + UI

- {isFavorite: true/false}
- Mise à jour favoriteIds[]
- Classe CSS active toggle
- Animation cœur

Récapitulatif des URLs du projet

URL	DESCRIPTION	ACCÈS
/	Site public — catalogue montres	Public

URL	DESCRIPTION	ACCÈS
/connexion	Login utilisateur	Public
/inscription	Création de compte	Public
/mon-espace	Espace client — favoris	User connecté
/admin/login	Login administrateur	Public
/admin/dashboard	Dashboard admin — CRUD montres	Admin
/admin/facturation	Module facturation complet	Admin

Identifiants Admin

Email : **admin@maisonxciv.com**
Mot de passe : **Xciv2025!**

Lancement du serveur

```
# Production
npm start

# Développement (nodemon)
npm run dev

# → http://localhost:3000
```

Points cles a retenir pour l'oral

- JSON DB = choix pragmatique (pas de build tools C++)
- 2 cookies séparés = sécurité et séparation des rôles
- optionalUserAuth = site public toujours accessible
- JWT = token autoporteur, serveur sans état (stateless)
- bcrypt = hachage lent intentionnel (protection brute-force)
- HttpOnly cookie = protection XSS
- initAuth() avant loadWatches() = ordre asynchrone
- Pattern MVC : Routes → Controllers → Models
- multer.diskStorage = sauvegarde images sur disque
- IntersectionObserver = animations performantes au scroll